Basic IPC

2026 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 7
2.0.0

Based on original slides by Professor Jon Eyolfson.

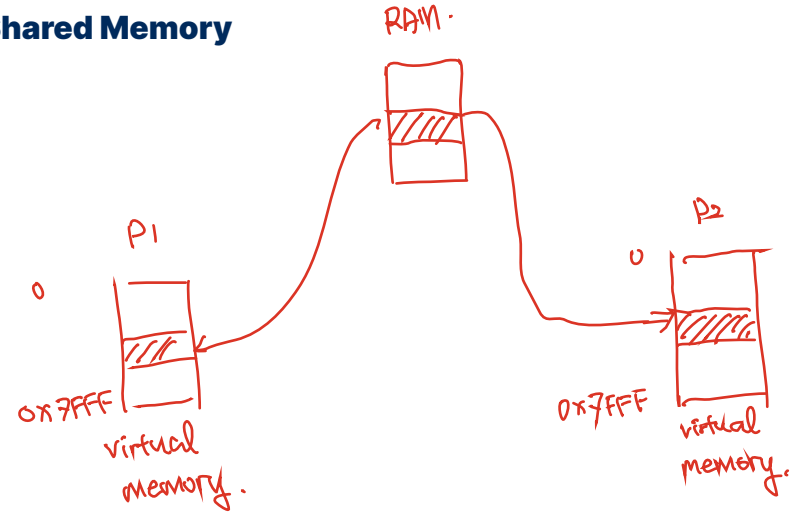
Interprocess Communication (IPC)

Processes do not share any memory with each other

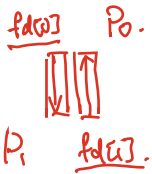
Some processes might want to work together for a task, so need to communicate information

IPC are mechanisms to share information between processes

Shared Memory



Pipe



A pipe is a half-duplex communication mechanism:

- Data written into ($fd[1]$) can be read from ($fd[0]$).
- Data flows in one direction only.

Regular pipes:

- Both file descriptors exist in the same process.
- The parent and child process share the pipe after a fork().
- The parent writes to the pipe, and the child reads from it (or vice versa).

Named pipes (FIFOs):

- A named pipe allows communication between unrelated processes.
- Exists as a special file in the filesystem.

resource.

Pipe Example

```c
int fd[2]; // File descriptors for the pipe
pipe(fd);

pid_t pid;
char write_msg[] = "Hello from parent!";
char read_msg[100];
pid = fork();

if (pid > 0) {
    write(fd[1], write_msg, strlen(write_msg) + 1);
} else {
    read(fd[0], read_msg, sizeof(read_msg));
    printf("Child received: %s\n", read_msg);
}
```

parent → write

child → read

parent & child share same fds.

fd[0] & [1] are both left open.

Pipe Example

```c
int fd[2]; // File descriptors for the pipe
pipe(fd);

pid_t pid;
char write_msg[] = "Hello from parent!";
char read_msg[100];
pid = fork();
if (pid > 0) {
    close(fd[0]); // close reading end.
    write(fd[1], write_msg, strlen(write_msg) + 1); // writing msg.
    close(fd[1]); // close writing end.
} else {
    close(fd[1]); // close writing end
    read(fd[0], read_msg, sizeof(read_msg)); // read from pipe.
    close(fd[0]); // close pipe read.
    printf("Child received: %s\n", read_msg);
}
```

parent →

child →

IPC is Transferring Bytes Between Two or More Processes

Reading and writing files is a form of IPC

The read and write system calls allow any bytes

Signals are a Form of IPC that Interrupts

You could also press Ctrl+C to stop a program
  This interrupts your programs execution and exits early

Kernel sends a number to your program indicating the type of signal
  Kernel default handlers either ignore the signal or terminate your process

Ctrl+C sends `SIGINT` (interrupt from keyboard)

If the default handler occurs the exit code will be 128 + signal number

You Can Set Your Own Signal Handlers with `sigaction`

You just declare a function that doesn't return a value, and has an `int` argument

The integer is the signal number

Some numbers are non-standard, below are a few from Linux x86-64:

- 2: `SIGINT` (interrupt from keyboard)
- 9: `SIGKILL` (terminate immediately)
- 11: `SIGSEGV` (memory access violation)
- 15: `SIGTERM` (terminate)

Most Operations Are Non-Blocking

A non-blocking call returns immediately, and you check if something occurs

To turn `wait` into a non-blocking call, use `waitpid` with `WNOHANG` in `options`

To react to changes to a non-blocking call, we can either use a *poll* or *interrupt*

Polling Continuously Calls the Function and Checks for Changes

We call `waitpid` over and over until the child exits

Note: some hardware behaves like this,
the kernel may have to check for changes

What's the drawback of this approach?

An Interrupt Instead Occurs Right After the Change

Instead of calling `wait` or `waitpid` from `main`, we can do it in the interrupt handler

- The kernel sends the `SIGCHLD` whenever one of its children exit

This idea also applies to the kernel, hardware can generate interrupts

Interrupt Handlers Run to Completion

An interrupt may occur while an interrupt handler is already running

All interrupt handler code must be reentrant
You need to be able to pause execution,
execute another call (to the same function),
and resume execution

On a RISC-V CPU, There's 3 Terms for "Interrupts"

Interrupt
Triggered by external hardware,
handled by the kernel (needs to respond quickly)

Exception
Triggered by an instruction (divide by zero, illegal memory access),
default handler is the kernel (calling process suspended),
the process can optionally handle some of these themselves

Trap
Transfer of control to a trap handler caused by either
an exception or an interrupt (code that runs)

A system call would be a *requested trap*

Review Question

```c
int main(){
    int fd[2];
    pid_t pid;
    char buffer[20];

    pid = fork();
    pipe(fd);

    if (pid == 0) {
        write(fd[1], "Hello", 6);
        printf("Child wrote to pipe\n");
    } else {
        read(fd[0], buffer, sizeof(buffer));
        printf("Parent received: %s\n", buffer);
    }
}
```